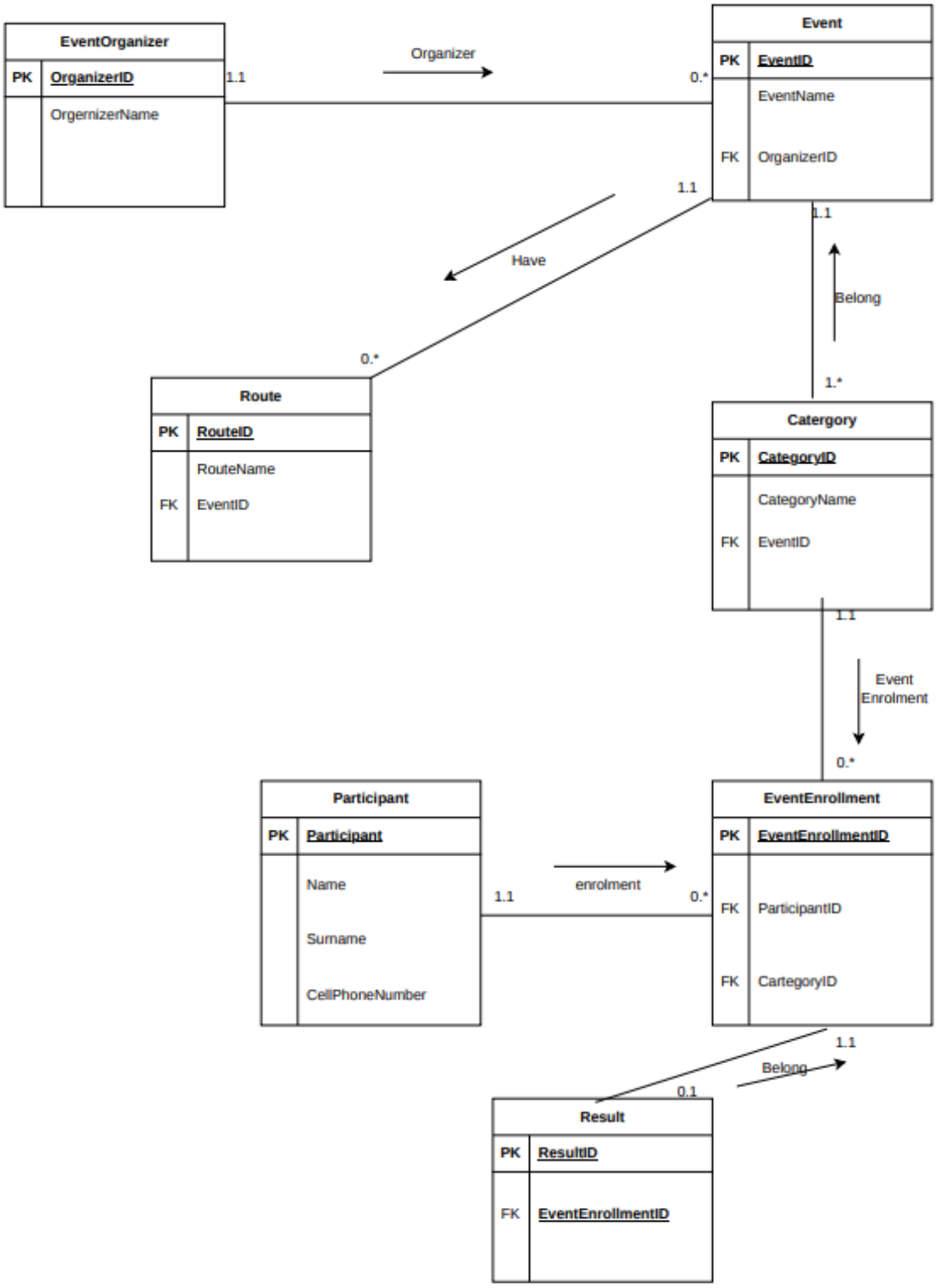
PROGRAMMING 2B

Name : Tlangelani
Surname : Shilaluke
Module code : PROG6212
Lecture : SHIBITI
Student Number: ST10479220

Section A

- **Role-Based Access Control (RBAC):** Users register with specific roles (Participant or Organiser). Certain actions (like creating events or recording results) are strictly restricted to Event Organisers.
- **Unique Identity:** Every user account must have a unique email address. Login requires valid credentials matched against securely hashed passwords, returning a JWT bearer token.
- **Organiser Ownership:** An Event Organiser can manage zero or many events, but **every event must have one and only one Event Organiser** as its owner.
- **CRUD Permissions:** Only the specific Event Organiser who created an event (or an Admin) can update or delete that event.
- **Strict Belonging:** A Route (e.g., 5km, 10km, 21km) must belong to **one and only one Event**. An event can have zero or multiple routes.
- **Management Control:** Routes can only be added, updated, or removed by the Event Organiser who owns the parent event.
- **Mandatory Categorisation:** An Event **must have at least one Category** (e.g., Senior Men, Open Women) and can have many. Every Category belongs to one and only one Event.
- **Deletion Restriction:** A Category **cannot be deleted** if there are active Event Enrolments linked to it (doing so triggers a 409 Conflict error to preserve data integrity).

- **Participant Registration:** A Participant can have zero or many Event Enrolments across different events and categories.
- **Enrolment Linkage:** Every Event Enrolment must link **one and only one Participant** to **one and only one Category** (and optionally a Route).
- **Duplicate Prevention:** Participants cannot duplicate active enrolments in the same category for the same event. Enrolments can be cancelled by the participant or organiser.
- **Optional Outcome:** An Event Enrolment can have **zero or one Result** (finish time, position, status). Conversely, every Result must belong to one and only one specific Event Enrolment.
- **Organiser Control:** Only Event Organisers can record, update, or delete finish times and race results.
- **Single Result Constraint:** An enrolment cannot have more than one result record; attempting to post a second result for the same enrolment results in a conflict.


EventOrganizer
PK
OrganizerID
OrgernizerName
1.1
Organizer
0.*
Event
PK
EventID
EventName
FK
OrganizerID
1.1
Have
1.1
Belong
0.*
1.*
Route
PK
RouteID
RouteName
FK
EventID
Catergory
PK
CategoryID
CategoryName
FK
EventID
1.1
Event
Enrolment
0.*
Participant
PK
Participant
Name
Surname
CellPhoneNumber
1.1
enrolment
0.*
EventEnrollment
PK
EventEnrollmentID
FK
ParticipantID
FK
CartegoryID
1.1
Belong
0.1
Result
PK
ResultID
FK
EventEnrollmentID

Section B

1.Authentication Endpoints

HTTP Method	Route	Description	Role Required	Request Body (if any	Expected Respons
POST	`/api/auth/register`	Register a new user (Organiser or Participant	Public	`{ name, email, password, role`	`201 Created`: User details & JWT token
POST	`/api/auth/logi`	Authenticate an existing user and return a token	Public	`{ email, password`	`200 OK`: JWT token & user summar

2. User Profile Endpoints

HTTP Method	Route	Description	Role Required	Request Body (if any)	Expected Response
Get	/api/users/profile	Retrieve the profile of the currently logged-in user	Authenticated User	None	200 OK: User profile object
PUT		Update the profile of the logged-in user	Authenticated User	`{ name, email, password? }`	`200 OK`: Updated profile object

3.Event Endpoints

HTTP Method	Route	Description	Role Required	Request Body (if any)	Expected Response
GET	/api/events	Retrieve a list of all available events	Public	None	200 OK: Array of event objects
GET	/api/events/{id}	Retrieve details for a specific event (including routes & categories)	Public	None	200 OK: Detailed event object
POST	/api/events	Create a new event (automatically linked to the logged-in organiser)	Event Organiser	{ title, description, date, location }	201 Created: Newly created event object
PUT	/api/events/{id}	Update details of a specific event	Event Organiser (Owner)	{ title, description, date, location }	200 OK: Updated event object
DELETE	/api/events/{id}	Delete a specific event	Event Organiser (Owner)	None	204 No Content: Deletion confirmation

4.Route Endpoints

HTTP Method	Route	Description	Role Required	Request Body (if any)	Expected Response
GET	/api/events/{eventId}/routes	Get all routes belonging to a specific event	Public	None	200 OK: Array of route objects
POST	/api/events/{eventId}/routes	Add a new route to a specific event	Event Organiser (Owner)	{ name, distance, elevationGain, description }	201 Created: Newly created route object
PUT	/api/routes/{id}	Update route details	Event Organiser (Owner)	{ name, distance, elevationGain, description }	200 OK: Updated route object
DELETE	/api/routes/{id}	Delete a route	Event Organiser (Owner)	None	204 No Content: Deletion confirmation

5. Category

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/events/{eventId}/categories	Retrieves all Categories belonging to an Event.	None (Public)	None	**200 OK** - Returns categories. **404 Not Found**.
POST	/api/events/{eventId}/categories	Creates a Category belonging to a specific Event.	Organiser	{"categoryName": "Open Men", "minAge": 18, "maxAge": 39}	**201 Created** - Returns category. **400 Bad Request**.
PUT	/api/categories/{id}	Updates an existing Category.	Organiser	{"categoryName": "Open Men (18-40)"}	**200 OK** - Returns updated category. **404 Not Found**.
DELETE	/api/categories/{id}	Deletes a Category.	Organiser	None	**204 No Content** - Deleted.

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
					- Has enrolments.

6. Event Enrolment Endpoints

HTTP Method	Route	Description	Role Required	Request Body (if any)	Expected Response
POST	/api/enrolments	Enrols the logged-in participant into a specific event category and route.	Participant	{ categoryId, routeId }	**201 Created**: Enrolment record created successfully. **400 Bad Request**: Already enrolled. **404 Not Found**: Category or route not found.
GET	/api/enrolments/my-enrolments	Retrieves all event enrolments for the	Participant	None	**200 OK**: Array of enrolme

HTTP Method	Route	Description	Role Required	Request Body (if any)	Expected Response
		currently logged-in participant.			nt objects.
GET	/api/categories/{categoryId}/enrolments	Retrieves all participant enrolments belonging to a specific category.	Event Organiser (Owner)	None	**200 OK**: Array of enrolment objects. **403 Forbidden**: Not event owner.
DELETE	/api/enrolments/{id}	Cancels/deletes an event enrolment.	Participant or Organiser		

7. Result Endpoints

HTTP Method	Route	Description	Role Required	Request Body (if any)	Expected Response
POST	/api/enrolments/{enrolmentId}/result	Records a finish time and position for a specific participant enrolment.	Event Organiser (Owner)	{ finishTime, position, status }	**201 Created**: Result record created. **409 Conflict**: Result already exists for this enrolment.
GET	/api/enrolments/{enrolmentId}/result	Retrieves the result associated with a specific enrolment.	Authenticated User	None	**200 OK**: Result object. **404 Not Found**: Result not yet recorded.

HTTP Method	Route	Description	Role Required	Request Body (if any)	Expected Response
PUT	/api/results/{id}	Updates an existing finish time or position.	Event Organiser (Owner)	{ finishTime, position, status }	**200 OK**: Updated result object.
DELETE	/api/results/{id}	Deletes a recorded result.	Event Organiser (Owner)		

Section C

```
create Database Race_DB;
use Race_DB;

create table EventOrganizer(
OrganizerID int primary key,
OrganizerName VarChar(255) not null
);

Insert into EventOrganizer
Values
(001, 'Mohau Nkota'),
(002, 'Tito Maswangani'),
(003, 'Oswin Appollis');



Create table Event(
EventID int Primary key,
EventName VarChar(255) not null,
OrganizerID int not null
Foreign key(OrganizerID) references EventOrganizer(OrganizerID)
);

Insert into Event
Values
(101, 'Scorpion kings', 001),
(102, 'Marshal', 003);


create table Category(
CategoryID int primary key,
CategoryName VarChar(255) not null,
EventID int not null,
Foreign key(EventID) references Event(EventID)
);

Insert into Category
Values
(5002, 'Rangers', 101),
(5003, 'Blacklist', 102);

Create table Route(
RouteID int Primary key,
RouteName Varchar(255),
EventID int not null,
Foreign key(EventID) references Event(EventID)
);
```

```
Insert into Route
values
(906, 'Mabena', 101),
(907, 'Ontella', 102);

Create table Participant(
ParticipantID int primary key,
Name Varchar(100) not null,
Surname Varchar(100) not null,
CellPhoneNumber Varchar(100) not null
);

insert into Participant
values
(6003, 'Gordens', 'Maluleke', '069 258 1456'),
(6005, 'Tony', 'Cipriani', '075 581 9435');

Create table EventEnrollment(
EventEnrollmentID int Primary key,
ParticipantID int not null,
CategoryID int not null,
Foreign Key(ParticipantID) references Participant(ParticipantID),
Foreign Key(CategoryID) references Category(CategoryID)
);

insert into EventEnrollment
values
(4001, 6003, 5002),
(4002, 6005, 5003);

Create table Result(
ResultID int Primary key,
EventEnrollmentID int not null,
Foreign key(EventEnrollmentID) references EventEnrollment(EventEnrollmentID)
);

insert into Result
Values
(9008, 4001),
(9009, 4002);

Select * from Result;
```

Reference List:

- Michael Goodwin, 2024 available at: https://www.ibm.com/think/topics/api
- SCRUMS.COM, 2026 available at:https://www.scrums.com/hire?gadid=813138814731&utm_source=google&utm_medium=paid&utm_campaign=23949331088&utm_content=197665395116&utm_term=software%20building&hsa_acc=2728752194&hsa_cam=23949331088&hsa_grp=197665395116&hsa_ad=813138814731&hsa_src=g&hsa_tgt=kwd-170314285&hsa_kw=software%20building&hsa_mt=b&hsa_net=adwords&hsa_ver=3&gad_source=1&gad_campaignid=23949331088&gbraid=0AAAAACeeHSjyWrbIMUUWeknheLb3tH2Mv&gclid=EAIaIQobChMIi_SwgIvVlgMVn5lQBh0Loyt9EAAYAiAAEgLmf_D_BwE
-